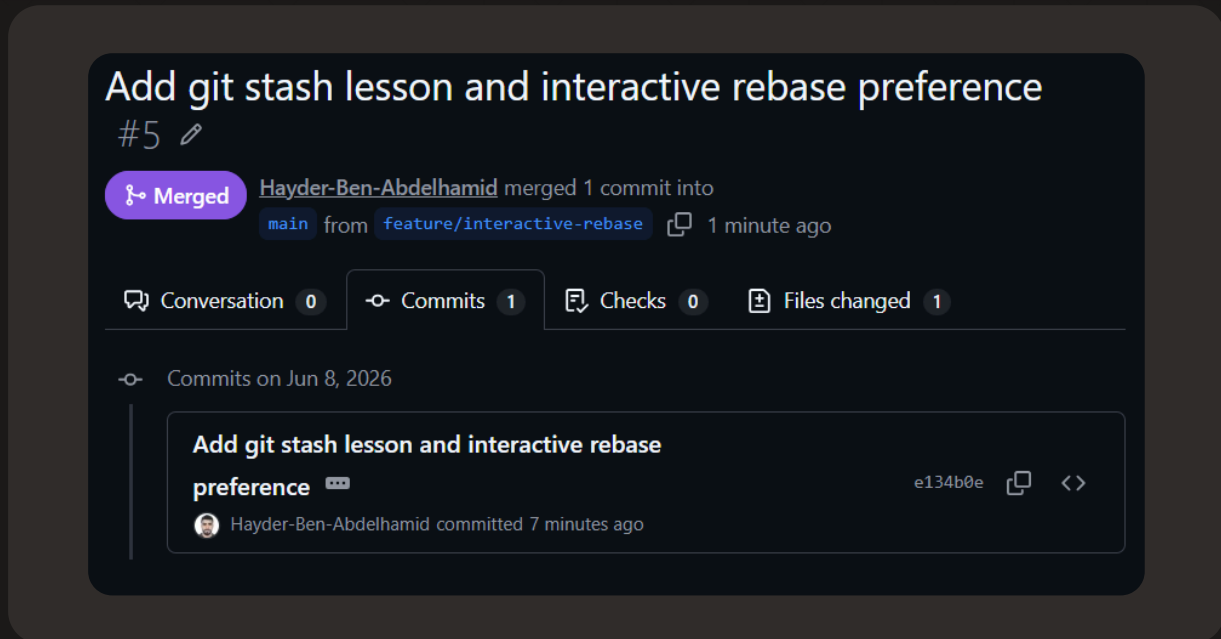




# GIT BRANCHING AND REBASING

By Hayder Ben Abdelhamid | June 2026



## Mastering Git for Professional Collaboration

### Project goals and motivation

In this project, I'm learning how to get really into git fundamentals like resolving merge conflicts, rebasing feature branch and cleaning up commit history so that i can be a professional team player with my git skills.

# Setting Up the Conflict Practice Environment

## Step overview

In this step, I'm setting up my environment and tools ready so that I can get into understanding merge conflicts and the importance of commit history.

```
git add conflict-practice.md
C:\Users\MSI\git-learning-log>
C:\Users\MSI\git-learning-log>git commit -m "Add conflict-practice.md"
[main 713bc39] Add conflict-practice.md
 1 file changed, 3 insertions(+)
 create mode 100644 conflict-practice.md

C:\Users\MSI\git-learning-log>git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 406 bytes | 406.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Hayder-Ben-Abdelhamid/git-learning-log.git
 66f1355..713bc39  main -> main

C:\Users\MSI\git-learning-log>
```

## Why create the file on main first

I created this file on main so that I have all the changes and I need in a simple manner that is on the main branch, Because I want to practice merge conflicts. I need to create branches that duplicate what's there in the main and then make changes so that I can merge it. That's the best way of really understanding what happens when there's merge conflicts.

# Resolving a Merge Conflict

## Step overview

In this step, I'm setting up Two branches that both edit the same line so that I can create a conflict, which I will the merge, resclive, and push to GitHub.

## Understanding conflict markers

The markers show me where he changes in the branches start and end,The <<<<<< marks Start of the current branches version. The ===== separates two different versions.The >>>>>> marks theend of the incoming branches.

```
C:\Users\MSI\git-learning-log>git log --oneline
e512cf9 (HEAD -> main, origin/main, origin/HEAD) Resolve merge conflict: combine workflow and t
ools sections
7509d60 (branch-b) Add tools section
d7853ca (branch-a) Add workflow preferences section
713bc39 Add conflict-practice.md
66f1355 Add pull request template
d2f0570 Add Next Steps section (#4)
f6269c6 Merge pull request #2 from Hayder-Ben-Abdelhamid/feature/1-add-resources
9a33fa2 Add resources section with helpful Git links
6fc87dc Merge add-resources branch and resolve conflict
f441b24 Rewrite learning notes with time machine analogy
0cdcc42 (origin/add-resources, add-resources) Expand learning notes on branches
edd530c Add What I Learned section
908a38a Add initial learning log

C:\Users\MSI\git-learning-log>
```

# Rebasing a Feature Branch for Clean Linear History

## Step overview

In this step, I'm setting up a new feature branch with new commits and simulate a teammate pushing to main. That way I can rebase my feature branch onto main's latest commit and understand how to get a clean linear history.

```
C:\Users\MSI\git-learning-log>git log --oneline
f7ae9ee (HEAD -> main, feature/rebase-practice) Add lessons learned section
da843fb Add team updates file
9c9a1f5 Add team updates file
e512cf9 (origin/main, origin/HEAD) Resolve merge conflict: combine workflow and tools sections
7509d60 (branch-b) Add tools section
d7853ca (branch-a) Add workflow preferences section
713bc39 Add conflict-practice.md
66f1355 Add pull request template
d2f0570 Add Next Steps section (#4)
f6269c6 Merge pull request #2 from Hayder-Ben-Abdelhamid/feature/1-add-resources
9a33fa2 Add resources section with helpful Git links
6fc87dc Merge add-resources branch and resolve conflict
f441b24 Rewrite learning notes with time machine analogy
0cdcc42 (origin/add-resources, add-resources) Expand learning notes on branches
edd530c Add What I Learned section
908a38a Add initial learning log
```

## Rebase vs. merge explained

Rebasing works by cleaning up all the commits into one so it overrides all the commit history. and that way you have one commit for all the comments that you've done and while merging works by I would choose rebase when i'm working alone on a branch and i don't wanna see all the commits when i push it into another branch where my teammate may be working.

# Polishing Commits with Interactive Rebase

## Step overview

In this step, I'm cleaning up my commit history, messy commits, bad commit messages in a clean professional history, convert it to a clean professional history using interactive rebase to reword commit message and squash commits together so that my history looks clean.

## reword vs. squash commands

Reword allows you to change the commit message in the commit history while squash combines the commits into one.

## Why --force-with-lease is required after rebase

I needed --force-with-lease because...

# Saving Work-in-Progress with Git Stash

```
C:\Users\MSI\git-learning-log>git add .

C:\Users\MSI\git-learning-log>git commit -m "Add stash lesson to conflict practice"
[stash-practice 6795574] Add stash lesson to conflict practice
1 file changed, 2 insertions(+), 1 deletion(-)

C:\Users\MSI\git-learning-log>git push origin stash-practice
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 340 bytes | 340.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'stash-practice' on GitHub by visiting:
remote:   https://github.com/Hayder-Ben-Abdelhamid/git-learning-log/pull/new/stash-practice
remote:
To https://github.com/Hayder-Ben-Abdelhamid/git-learning-log.git
* [new branch]      stash-practice -> stash-practice

C:\Users\MSI\git-learning-log>
```

## The value of git stash in team workflows

In this project extension, I learned that git stash helps when you're working through a file, but you suddenly need to go to another branch midway through your work. You can stash it to save the work and come back to it.